

# Pramo Jewels

## AI Backend Coding Guide

Document ID: PJ-AIBCG-001  
Version: 1.0  
Status: Approved

| Field    | Value                                                                        |
|----------|------------------------------------------------------------------------------|
| Project  | Pramo Jewels Premium E-Commerce Platform                                     |
| Purpose  | Define backend development standards for AI-generated server-side code, APIs |
| Audience | Backend Developers, Solution Architects, AI Coding Assistants, DevOps, QA    |
| Version  | 1.0                                                                          |

### AI Backend Development Standards

| Category       | Guidelines                                                      |
|----------------|-----------------------------------------------------------------|
| Architecture   | Follow layered architecture and separation of concerns          |
| REST APIs      | Consistent endpoints, validation, versioning and error handling |
| Business Logic | Encapsulate logic in services; keep controllers lightweight     |
| Database       | ORM usage, migrations, transactions and indexed queries         |
| Security       | RBAC, JWT/session auth, input validation, encryption            |
| Performance    | Caching, pagination, async jobs and optimized queries           |
| Logging        | Structured logs, correlation IDs and audit events               |
| Testing        | Unit, integration and API contract tests                        |

### Objectives

Ensure AI-generated backend code is secure, scalable, maintainable and aligned with the approved system architecture.

### Coding Standards

Use descriptive naming, dependency injection, modular services, reusable utilities and project-defined folder conventions.

### API Guidelines

Validate requests, return standardized responses, implement idempotency where applicable and document endpoints.

### Security Requirements

Prevent SQL injection, XSS and CSRF where relevant, never hard-code secrets, enforce authorization and sanitize all inputs.

## **Performance Guidelines**

Optimize database access, avoid N+1 queries, use caching, queues and background processing for long-running tasks.

## **Documentation**

Document APIs, services, database changes, configuration updates and business rules with every implementation.

## **Integration**

Integrates with Authentication, Database, Cache, Payment Gateway, Shipping APIs, CMS, Analytics, Monitoring and CI/CD.

## **Acceptance Criteria**

AI-generated backend code passes code review, security validation, automated tests, performance benchmarks and conforms to project standards.